Floating Point

CSE 238/2038/2138: Systems Programming LAB

Instructor:
Fatma CORUT ERGİN

Teaching Assistants:
Lokman ALTIN – Zuhal ALTUNTAŞ

Floating Point in C

- C Guarantees Two Levels
  - float single precision
  - double double precision
- Conversions/Casting
  - Casting between int, float, and double changes bit representation
  - double/float → int
    - Truncates fractional part
    - Like rounding toward zero
    - Not defined when out of range or NaN: Generally sets to TMin
  - int/float → double
    - Exact conversion, as long as int has ≤ 53 bit word size
  - int → float
    - Will round according to rounding mode

Floating Point C Puzzles

Initialization

```
int x = …;
float f = …;
double d = …;
```

Assume
neither
d nor f is NaN

`x == (int)(float) x`	

Floating Point C Puzzles

Initialization

```
int x = …;
float f = …;
double d = …;
```

Assume
neither
d nor f is NaN

`x == (int)(float) x`	**False**
`x == (int)(double) x`	

Floating Point C Puzzles

Initialization

```
int x = …;
float f = …;
double d = …;
```

Assume
neither
d nor f is NaN

`x == (int)(float) x`	False
`x == (int)(double) x`	True
`f == (float)(double) f`	

Floating Point C Puzzles

Initialization

```
int x = …;
float f = …;
double d = …;
```

Assume
neither
d nor f is NaN

`x == (int)(float) x`	False
`x == (int)(double) x`	True
`f == (float)(double) f`	True
`d == (double)(float) d`	

Floating Point C Puzzles

Initialization

```
int x = …;
float f = …;
double d = …;
```

Assume
neither
d nor f is NaN

`x == (int)(float) x`	False
`x == (int)(double) x`	True
`f == (float)(double) f`	True
`d == (double)(float) d`	False
`f == -(-f);`	

Floating Point C Puzzles

Initialization

```
int x = …;
float f = …;
double d = …;
```

Assume
neither
d nor f is NaN

`x == (int)(float) x`	False
`x == (int)(double) x`	True
`f == (float)(double) f`	True
`d == (double)(float) d`	False
`f == -(-f);`	True
`2/3 == 2/3.0`	

Floating Point C Puzzles

Initialization

```
int x = …;
float f = …;
double d = …;
```

Assume
neither
d nor f is NaN

`x == (int)(float) x`	False
`x == (int)(double) x`	True
`f == (float)(double) f`	True
`d == (double)(float) d`	False
`f == -(-f);`	True
`2/3 == 2/3.0`	False
`d < 0.0 ⇒ ((d*2) < 0.0)`	

Floating Point C Puzzles

Initialization

```
int x = …;
float f = …;
double d = …;
```

Assume
neither
d nor f is NaN

`x == (int)(float) x`	False
`x == (int)(double) x`	True
`f == (float)(double) f`	True
`d == (double)(float) d`	False
`f == -(-f);`	True
`2/3 == 2/3.0`	False
`d < 0.0 ⇒ ((d*2) < 0.0)`	True
`d > f ⇒ -f > -d`	

Floating Point C Puzzles

Initialization

```
int x = …;
float f = …;
double d = …;
```

Assume
neither
d nor f is NaN

`x == (int)(float) x`	False
`x == (int)(double) x`	True
`f == (float)(double) f`	True
`d == (double)(float) d`	False
`f == -(-f);`	True
`2/3 == 2/3.0`	False
`d < 0.0 ⇒ ((d*2) < 0.0)`	True
`d > f ⇒ -f > -d`	True
`d*d >= 0.0`	

Floating Point C Puzzles

Initialization

```
int x = …;
float f = …;
double d = …;
```

Assume
neither
d nor f is NaN

`x == (int)(float) x`	False
`x == (int)(double) x`	True
`f == (float)(double) f`	True
`d == (double)(float) d`	False
`f == -(-f);`	True
`2/3 == 2/3.0`	False
`d < 0.0 ⇒ ((d*2) < 0.0)`	True
`d > f ⇒ -f > -d`	True
`d*d >= 0.0`	True
`(d+f)-d == f`	

Floating Point C Puzzles

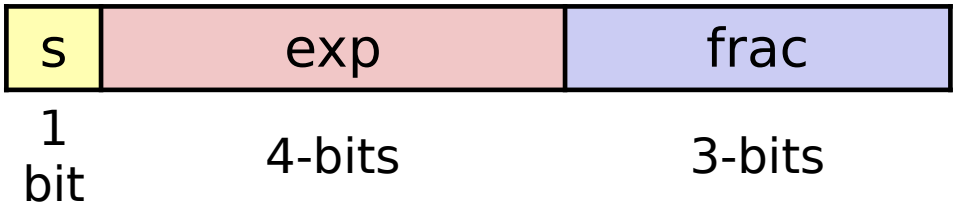
Initialization

```
int x = …;
float f = …;
double d = …;
```

Assume
neither
d nor f is NaN

`x == (int)(float) x`	False
`x == (int)(double) x`	True
`f == (float)(double) f`	True
`d == (double)(float) d`	False
`f == -(-f);`	True
`2/3 == 2/3.0`	False
`d < 0.0 ⇒ ((d*2) < 0.0)`	True
`d > f ⇒ -f > -d`	True
`d*d >= 0.0`	True
`(d+f)-d == f`	False

Creating Floating Point Number



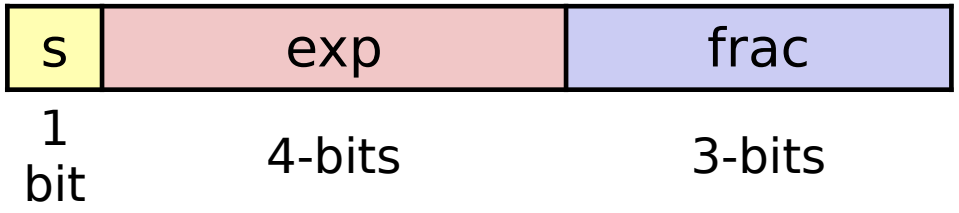
- Steps
  1. Normalize to have leading 1
  2. Round to fit within fraction
  3. Postnormalize to deal with effects of rounding

- Case Study: Convert numbers to tiny floating point format

Value	Binary
128	10000000_2
15	00001111_2
33	00100001_2
35	00100011_2
63	00111111_2

Step 1: Normalize to have leading 1

- Requirement
  - Set binary point so that numbers of form 1.xxxxx
  - Adjust all to have leading one
    - Decrement exponent as shift left

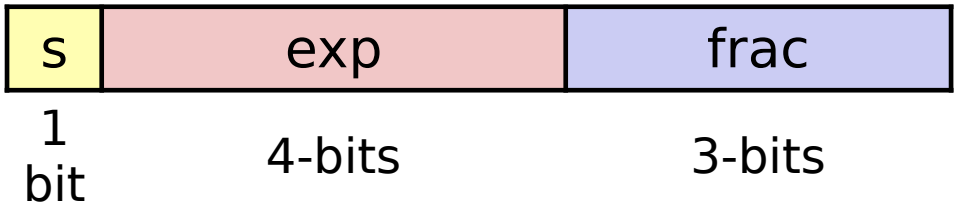


Value	Binary	Fraction	Exponent
128	10000000_2	1.0000000_2	7
15	00001111_2	1.1110000_2	3
33	00100001_2	1.0000100_2	5
35	00100011_2	1.0001100_2	5
63	00111111_2	1.1111100_2	5

Step 2: Round to fit within fraction

Round-to-nearest-even

frac = 3 bits



Value	Fraction	Rounded	Exponent
128	1.0000000_2	1.000_2	7
15	1.1110000_2	1.111_2	3
33	1.0000100_2	1.000_2	5
35	1.0001100_2	1.001_2	5
63	1.1111100_2	10.000_2	5

Step 3: Postnormalize to deal with effects of rounding

- Issue
  - Rounding may have caused overflow
  - Handle by shifting right once & incrementing exponent

Value	Rounded	Exponent	Adjusted	Adjusted Exponent	Result
128	1.000_2	7		7	$1.000_2 * 2^7 = 128$
15	1.111_2	3		3	$1.111_2 * 2^3 = \frac{15}{8} * 2^3 = 15$
33	1.000_2	5		5	$1.000_2 * 2^5 = 32$
35	1.001_2	5		5	$1.001_2 * 2^5 = \frac{9}{8} * 2^5 = 36$
63	10.000_2	5	1.000_2	6	$1.000_2 * 2^6 = 64$

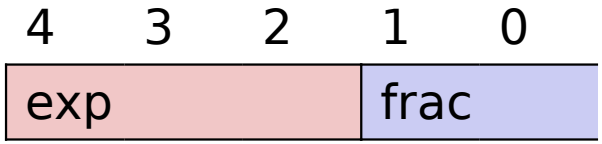
Binary Representations

$$value = (-1)^s \; M \; 2^E$$
$$E = e - Bias$$
$$Bias = 2^{k-1} - 1 = 7$$

Unsigned Value	Mantissa	Exponent	Floating Point Value	Binary representation
128	1.000_2	7	128	0 1110 000
15	1.111_2	3	15	0 1010 111
33	1.000_2	5	32	0 1100 000
35	1.001_2	5	36	0 1100 001
63	1.000_2	6	64	0 1101 000

Floating Point – Example

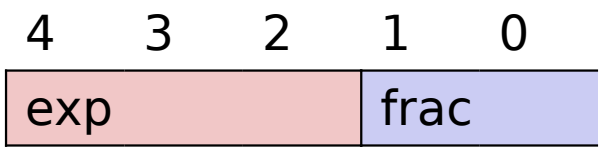
- Consider the following 5-bit floating point representation based on the IEEE floating point format. This format does not have a sign bit – it can only represent nonnegative numbers.
  - There are k=3 exponent bits.
  - There are n=2 fraction bits.



- What is the…
  - Bias?
  - Largest denormalized number?
  - Smallest normalized number?
  - Largest finite number it can represent?
  - Smallest non-zero value it can represent?

Floating Point – Example

- Consider the following 5-bit floating point representation based on the IEEE floating point format. This format does not have a sign bit – it can only represent nonnegative numbers.
  - There are k=3 exponent bits.
  - There are n=2 fraction bits.



- What is the…
  - Bias? **011_2 = 3**
  - Largest denormalized number? **$000\ 11_2$ = 0.0011_2 = 3/16**
  - Smallest normalized number? **$001\ 00_2$ = 0.0100_2 = 1/4**
  - Largest finite number it can represent? **$110\ 11_2$ = 1110.0_2 = 14**
  - Smallest non-zero value it can represent? **$000\ 01_2$ = 0.0001_2 = 1/16**

Floating Point – Example

- For the same problem, complete the following table:

Value	Floating Point Bits	Rounded Value
9/32		
8		
9		
	000 10	
19		

Floating Point – Example

- For the same problem, complete the following table:

Value	**Floating Point Bits**	**Rounded Value**
9/32	**001 00**	**1/4**
8	**110 00**	**8**
9	**110 00**	**8**
1/8	000 10	
19	**111 00**	**inf**

Floating Point Recap

- Floating point = $(-1)^s$ M 2^E
- MSB is sign bit s
- Bias = $2^{(k-1)} - 1$ (k is num of exp bits)
- Normalized (larger numbers, denser towards 0)
  - exp ≠ 000…0 and exp ≠ 111…1
  - M = 1.frac
  - E = exp - Bias
- Denormalized (smaller numbers, evenly spread)
  - exp = 000….0
  - M = 0.frac
  - E = - Bias + 1

Floating Point Recap

- Special Cases
  - +/- Infinity: `exp` = 111…1 and `frac` = 000…0
  - +/- NaN: `exp` = 111…1 and `frac` ≠ 000…0
  - +0: `s` = 0, `exp` = 000…0 and `frac` = 000…0
  - -0: `s` = 1, `exp` = 000…0 and `frac` = 000…0
- Round towards even when half way (lsb of `frac` = 0)

Examples

- What is the 32-bit IEEE floating point representation for the following numbers?
  1. 5 3/4:
  2. 2 7/8:
  3. 30.125:

- With the 8 exponent bits of single-precision float, what is the smallest non-zero number with 0x01 as the exponent bits? Is this value greater or less than one? Express your answer as a power of 2.

Examples

- What is the 32-bit IEEE floating point representation for the following numbers?
  1. 5 3/4: 0 10000001 01110000000000000000000
  2. 2 7/8: 0 10000000 01110000000000000000000
  3. 30.125: 0 10000011 11100010000000000000000

- With the 8 exponent bits of single-precision float, what is the smallest non-zero number with 0x01 as the exponent bits? Is this value greater or less than one? Express your answer as a power of 2.

  0 00000001 00000000000000000000000 is the smallest non-zero number

  = $1.0 * 2^{1-127} = 2^{-126}$ -> smaller than one

Examples

- For a single-precision floating point number, using your new knowledge about denormalized form, what is the smallest positive (> 0) number that you can represent? Give your answer as a power of 2.

  0 00000000 00000000000000000000001 is the smallest positive number

  $= 0.00000000000000000000001 * 2^{1-127} = 2^{-23} * 2^{-126} = 2^{-149}$

- Compute the sum of the following binary numbers:
  - $1.010 * 2^2 + 1.110 * 2^3$

- Consider two different 6-bit floating point formats based on the IEEE floating point format given below:
  - **Format A:**
    - There is sign bit in the most significant bit.
    - The next 3 bits are the exponent. The bias is $2^{k-1} - 1 = 3$.
    - The last 2 bits are the fraction.
  - **Format B:**
    - There is sign bit in the most significant bit.
    - The next 2 bits are the exponent. The bias is $2^{k-1} - 1 = 1$.
    - The last 3 bits are the fraction.
- For formats A and B, fill in the empty boxes in the table below (use round-to-even when necessary).

Value	Format A Bits	Format B Bits	Explanation
Zero	0 000 00	0 00 000	
1			
	1 010 00		
		0 01 011	
7/2			

- Consider two different 6-bit floating point formats based on the IEEE floating point format given below:
  - **Format A:**
    - There is sign bit in the most significant bit.
    - The next 3 bits are the exponent. The bias is $2^{k-1} - 1 = 3$.
    - The last 2 bits are the fraction.
  - **Format B:**
    - There is sign bit in the most significant bit.
    - The next 2 bits are the exponent. The bias is $2^{k-1} - 1 = 1$.
    - The last 3 bits are the fraction.
- For formats A and B, fill in the empty boxes in the table below (use round-to-even when necessary).

Value	Format A Bits	Format B Bits	Explanation
Zero	0 000 00	0 00 000	
1	**0 011 00**	**0 01 000**	Exact in both formats
-1/2	1 010 00	**1 00 100**	Exact in both formats, norm in A, denorm in B
11/8	**0 011 10**	0 01 011	Format A round to even, format B exact
7/2	**0 100 11**	**0 10 110**	Exact in both formats